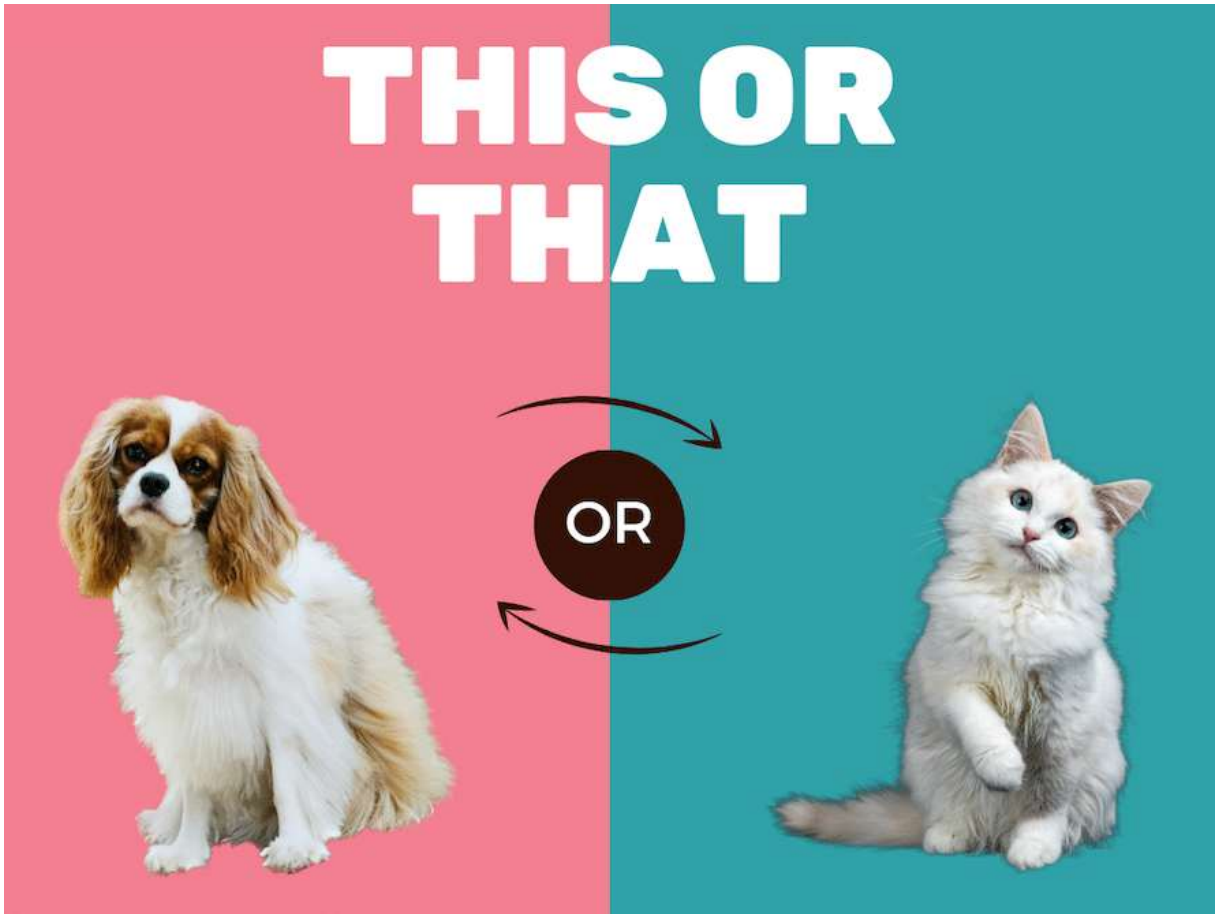
Task 15: optimization techniques

By Korneel Loy



Introduction : training a model

To explain optimization techniques, let's start at the beginning, explaining some basic concepts:

Machine learning is essentially giving a set of data, combined with a set of labels, to a machine, so the machine can find patterns within this data. Once it has learned these patterns, we can use the model to classify new, unseen data.

A simple example is a list of animal pictures, where each picture is represented as a vector of 2048 numbers where each number represents a visual characteristic (like texture, color or shapes), combined with the knowledge of what kind of animal is represented in each picture.

So what we do is train a model: we give it the training data (combined with the labels) and the model tries to find patterns, looking at the data over and over again, assigning a **weight** to each characteristic. For example, if a moustache is identified, the weight of that vector element will be high, and will likely get higher and higher as the model sees more and more pictures during training, because this characteristic is quite deterministic in telling whether a picture represents a cat rather than a dog.

Starting off with random weights per neuron, the model sees the same pictures over and over again, and adjusts the relative weights at each pass (called an **epoch**). With each epoch, the patterns become more refined. But the model doesn't just adjust weights. It also maintains a **bias** for each neuron, which is like a "default opinion" before it even looks at the data. Imagine a dataset with picture of dogs and cats. But one picture is verry blurry, and the data doesn't contain any identifiable characteristics. But we do know that 70% of the picture depict a dog, and 30% depict a cat. So the chances are that the blurry picture depicts a dog.

To know how well it's doing at each epoch, the model calculates a **loss**: a single number that measures how far its predictions are from the correct labels. A high loss means the model is getting a lot wrong. A low loss means it's getting close. The whole point of training is to minimize this loss, epoch after epoch, by continuously adjusting the weights and biases.

In the first epoch, the loss will be high, because the model is basically taking a wild guess. By the end of that first pass, the model starts to detect some patterns, and assigns initial weights and biases to the characteristics. In the second epoch, it uses those weights to make new predictions and checks whether the loss has decreased. If it has, the adjustments were in the right direction. If not, the model adjusts the weights and biases in the opposite direction.

In short : training a model is gradually finding the weight and the bias, limiting the loss, run by run. But there are ways to improve this training, optimizing it (see next page).

Optimizing a model

Optimizing a model can be done via different techniques:

Feature Scaling:

Features can be very different in nature. Does the animal have a moustache? 1 for yes, 0 for no. What is the color of the fur? 3 RGB values, each between 0 and 255. If we don't scale these features, the color values will have a much higher impact on the model simply because they are numerically larger, not because they are more meaningful. The solution is to bring all features to the same scale (for example, all values between 0 and 1).

Batch normalization:

Batch normalization can be thought of as a kind of feature scaling, but applied at each layer transition. Imagine the first layer gives a high weight to the moustache feature because it's very deterministic, and a low weight to the color feature because it doesn't tell much. The outputs of that layer (the values passed to the next one) can end up all over the place as a result. Batch normalization re-scales those outputs back into a scale.

Mini-batch gradient descent:

Gradient descent is the basic algorithm for training a model. At each epoch, the model makes predictions and checks in which direction the weights and biases need to be adjusted for the next round. In the basic version, the model looks at the entire dataset before adjusting the weights. In the mini-batch version, the dataset is divided into smaller subsets, and the weights are adjusted after each subset. The advantage is that learning is faster, but each adjustment is based on a smaller sample, so the path toward the minimum is a little noisier. At the start of each epoch, the dataset is reshuffled randomly before being divided into batches again.

Gradient descent with momentum:

This is another improvement on the basic algo. The idea is simple: instead of adjusting the weights based only on the current batch, the model also takes into account the direction it has been moving in previous batches. If the model has been consistently moving in the same direction, it builds up speed in that direction, like a ball rolling down a hill that gets faster as it goes. If the direction starts changing, the accumulated speed acts as a brake, smoothing out the zigzagging.

For example, imagine the model is training on our cat/dog dataset. For the last 10 batches, it has been consistently increasing the weight of the moustache feature, because it keeps reducing the loss. With momentum, the model says: "this direction has been working well for a while, let me take a bigger step in the same direction this time." On the other hand, if the weight of the color feature has been zigzagging, the model takes smaller, more cautious steps on that feature.

RMSProp optimization:

Like momentum, this improvement also adjusts the steps the model takes when updating the weights, but not based on the history of the gradient, but based on how frequently and strongly a feature appears in the data. For instance, nearly every picture gives a clear signal on whether the animal has a moustache or not. But information about tail size is simply not present in many pictures. For the moustache feature, the model gets so much feedback that it risks overshooting/overcorrection, so RMSProp takes smaller, more cautious steps. For tail size, the model rarely gets useful information, so when it does, RMSProp takes bigger steps to make the most of it.

Adam optimization:

Combination of momentum and RMSProp.

Learning rate decay:

The basic idea is simple: in the beginning, the model still has a lot to learn and is far from the optimal weights, so it makes sense to take big steps. But as training progresses and the model gets closer to the minimum, big steps become a problem, and the balance will be difficult to find. Learning rate decay solves this by deliberately slowing down the learning rate over time.